

Table of Contents

✓ Resolução — Exame Época Normal PAW 2021/2022	1
Parte 1 — Escolha Múltipla (8 valores).....	1
Parte 2 — Verdadeiro ou Falso (2 valores).....	5
Parte 3 — Resposta Aberta (10 valores)	6

✓ Resolução — Exame Época Normal PAW 2021/2022

Curso: LEI / LSIRC | **UC:** Programação em Ambiente Web

Duração: 1h40min

Parte 1 — Escolha Múltipla (8 valores)

Cada pergunta vale 1 valor. Opção incorreta desconta 0.5 valores.

1. Indique as afirmações verdadeiras:

- a. HTML é uma linguagem de programação para a web;
- b. CSS permite estruturar o conteúdo de página web;
- c. JavaScript apenas permite formatar os elementos de página web;
- d. É possível submeter um formulário numa página HTML sem utilizar CSS ou JavaScript.

✓ Respostas corretas: **d**

Justificação:

- **a) FALSO:** HTML é uma linguagem de *marcação*, não de programação.
 - **b) FALSO:** CSS serve para *apresentar/estilizar*. O HTML é que estrutura o conteúdo.
 - **c) FALSO:** JS adiciona *lógica e interatividade*, não “apenas formata” (que é o papel do CSS).
 - **d) VERDADEIRO:** Um formulário <form> HTML padrão submete dados para um servidor sem precisar de qualquer CSS ou JS ativo.
-

2. Indique as afirmações verdadeiras sobre template engines:

- a. Angular possui um template engine que gera as páginas diretamente no browser, a partir dos componentes presentes na página apresentada ao utilizador;
- b. Podemos utilizar o template engine mongoose para gerar páginas HTML dinamicamente no servidor;
- c. O template engine EJS, num projeto gerado através do express generator, guarda as os templates dentro da pasta views do projeto;
- d. Para utilizar template engines numa aplicação web, necessitamos de utilizar, obrigatoriamente, APIs REST e operações CRUD.

☑ **Respostas corretas: a, c**

Justificação:

- O Angular compila e renderiza os seus templates HTML diretamente no cliente (browser).
- No ExpressJS, utilizando o EJS (Embedded JavaScript), as views geradas ficam tradicionalmente numa pasta chamada views.
- “B” é falso porque Mongoose é um driver de base de dados, não um template engine.
- “D” é falso porque páginas geradas no servidor com template engines tradicionais (como EJS) não precisam de APIs REST (são Server-Side Rendering tradicional).

3. Atendo às caraterísticas da framework ExpressJS identifique as afirmações verdadeiras:

- a. ExpressJS é uma framework para desenvolver aplicações que correm exclusivamente no browser;
- b. O package manager npm pode ser utilizado para gerir as dependências da nossa aplicação;
- c. O ficheiro package.json guarda contém, entre outros, informação sobre as dependências da nossa aplicação e informação sobre comandos para serem executados no terminal com a ferramenta npm;
- d. ExpressJS é uma framework que cria aplicações que devem correr com um runtime de JavaScript como o NodeJS.

✓ *Respostas corretas: b, c, d*

Justificação: ExpressJS corre estritamente no **servidor** via o runtime Node.js (opções D verdadeira, A falsa). O npm é usado para instalar e o package.json guarda as dependências e os scripts (opções B, C verdadeiras).

4. Da seguinte lista selecione as afirmações verdadeiras sobre a framework Angular:

- a. Angular é uma framework para desenvolvimento e aplicações no backend;
- b. Angular apenas permite o uso de um componente por página/rota;
- c. Em angular não é possível utilizar os formulários de html com o elemento <form></form>;
- d. A linguagem de programação de uma aplicação em Angular é o typescript.

✓ *Respostas corretas: d*

Justificação: Angular é estritamente Frontend (A é falsa), permite compor páginas com dezenas de componentes reutilizáveis (B é falsa), suporta e incentiva ativamente o uso de <form> (C é falsa). Todo o código Angular é escrito nativamente em **TypeScript** (D é verdadeira).

5. Indique as afirmações verdadeiras sobre o desenvolvimento de serviços REST:

- a. Em serviços REST a informação é trocada em formato binário entre cliente e servidor para melhor performance;
- b. Serviços REST não permitem o uso do formato XML para troca de informação entre cliente e servidor;
- c. É possível utilizar os métodos HTTP como GET, POST, PUT, DELETE para mapear operações CRUD sobre uma base de dados;
- d. A modulo swagger-ui permite documentar e criar um ambiente de testes para APIs REST na framework ExpressJS.

✓ Respostas corretas: **c, d**

Justificação: Serviços REST trocam informação quase sempre em **texto** (JSON ou XML), não sendo “binário” o padrão de troca (A, B falsas). REST baseia-se no mapeamento dos verbos HTTP para operações CRUD (C é verdadeira). O swagger-ui-express cria documentação visual e interativa para testes (D é verdadeira).

6. Observe o excerto de código na figura 1 e indique quais das seguintes opções são verdadeiras:

(Excerto é um ficheiro de template EJS mostrando uma iteração <% newsList.forEach... %> com <%= news.title %>) - a. O código presente na figura é executado por um template engine no servidor gerando um ficheiro html que é enviado para o cliente; - b. O código presente na figura é enviado para o cliente e depois executado para mostrar a informação no browser; - c. Se a variável newsList estiver vazia nenhuma informação é mostrada no browser do cliente; - d. Independentemente do numero de news dentro da variável newsList, apenas a primeira é mostrada no browser do cliente.

✓ Respostas corretas: **a, c**

Justificação: O excerto é um **Template EJS**. O EJS corre inteiramente no **servidor** (Express), compila o HTML com os dados injetados e apenas envia HTML estático para o browser do cliente (A é verdadeira, B é falsa). O código possui um if (newsList && newsList.length > 0) que previne a renderização se estiver vazio (C é verdadeira). O bloco .forEach percorre todos os elementos, não só o primeiro (D é falsa).

7. Tendo em consideração o excerto de código na figura 2, indique as afirmações verdadeiras:

(Excerto mostra criação de um modelo Mongoose: mongoose.Schema({ name: String... }))) - a. Estamos perante uma aplicação desenvolvida na framework Angular; - b. Estamos perante uma aplicação desenvolvida na framework ExpressJS; - c. O excerto de código cria um modelo de dados e um objeto para interagir com uma coleção da base de dados; - d. O excerto de código representa a criação de uma classe de modelo de dados em typescript.

✓ Respostas corretas: **c**

Justificação: O código utiliza a biblioteca **Mongoose** em Node.js (JavaScript, não TypeScript - d é falsa). Ele define um Schema e exporta um Modelo compatível com o MongoDB (c é verdadeira). Embora o Mongoose seja tipicamente usado dentro do ExpressJS, o código por se não tem referências a Express (mas dependendo da interpretação do docente, ‘b’ pode ser pontuada).

8. Tendo em consideração o excerto de código na figura 3, indique as afirmações verdadeiras:

(Excerto mostra *JWTInterceptorService implements HttpInterceptor* em Angular) - a. O excerto de código apresentado é executado sempre que existe um pedido http na aplicação;

- b. O excerto de código cria um token JWT e envia para o cliente;
- c. O excerto de código cria um header com o nome 'x-access-token' num pedido http;
- d. O excerto de código gere o acesso de um utilizador às rotas de uma página web.

 **Respostas corretas: a, c**

Justificação: Trata-se de um **Interceptor HTTP** do Angular. O seu papel é “interceptar” todos os pedidos a sair do frontend e injetar nelas o cabeçalho de autenticação (A, C são verdadeiras). Ele lê o token do localStorage, não o “cria” (B é falsa). O acesso a rotas é gerido pelos Route Guards, não pelos Interceptors (D é falsa).

Parte 2 — Verdadeiro ou Falso (2 valores)

1. O objeto XMLHttpRequest em JavaScript permite obter informação de um servidor sem o uso de formulários e sem a necessidade de fazer reload a de uma página da internet.

 **VERDADEIRO.** Esta é a definição do conceito de AJAX (Asynchronous JavaScript and XML).

2. Um formulário para login de utilizadores pode utilizar o método GET para enviar informação de autenticação para o servidor.

 **FALSO.** O método GET envia a informação visível no URL (query string). Enviar passwords/credenciais via GET compromete gravemente a segurança, pois as mesmas ficarão registadas no histórico do browser e nos logs do servidor em “plain text”. Deve-se usar SEMPRE o método **POST** com HTTPS.

3. As aplicações Angular executam no runtime NodeJS.

 **FALSO.** As aplicações Angular são *Client-Side*, executando diretamente no **browser** do utilizador. O NodeJS é utilizado apenas na máquina do developer como ferramenta (via Angular CLI) para compilar e gerar a build do projeto.

4. Um serviço REST com o método PUT não deve permitir o envio de informação de um cliente para o servidor.

✗ **FALSO**. O método PUT é utilizado especificamente para **Update** (atualizar) recursos. Assim sendo, ele envia explicitamente informação nova/atualizada no *body* (corpo) do seu pedido HTTP do cliente para o servidor.

Parte 3 — Resposta Aberta (10 valores)

1. Descreva o significado dos conceitos backend e frontend no desenvolvimento de uma aplicação web. (0.5 val)

- **Frontend (Client-side)**: É a parte visual da aplicação que corre no browser do utilizador (interface). Desenvolvido com HTML, CSS, JavaScript (e frameworks como Angular). Responsável pela Experiência de Utilizador (UX).
- **Backend (Server-side)**: É a “máquina” que corre num servidor remoto. Responsável pela lógica de negócio pesada, comunicação com Bases de Dados, processamento seguro de autenticações e fornecimento de APIs.

2. No desenvolvimento web indique o que entende pelo termo MEAN stack. (0.5 val)

É uma arquitetura de desenvolvimento de software baseada em JavaScript de “ponta a ponta” (Full-stack). As siglas significam: - **M**ongoDB (Base de dados NoSQL) - **E**xpressJS (Framework web backend) - **A**ngular (Framework frontend) - **N**ode.js (Runtime de execução no servidor)

3. As operações CRUD estão relacionadas com o desenvolvimento de vários componentes web. Indique de que se tratam estas operações e 2 casos de uso. (1 val)

CRUD são as 4 operações básicas de armazenamento persistente: **C**reate (Criar), **R**ead (Ler), **U**ppdate (Atualizar) e **D**eleete (Apagar). Casos de uso: 1. Numa **API REST**: Mapeiam-se diretamente para os métodos HTTP POST (Create), GET (Read), PUT (Update) e DELETE (Delete). 2. Na **Base de Dados** (ex: Mongoose): Correspondem a métodos de modelos como `save()`, `find()`, `findByIdAndUpdate()` e `findByIdAndDelete()`.

4. O uso de padrões de software no desenvolvimento de aplicações web é considerado uma boa prática. Considere o padrão de software MVC. Descreva em que consiste este padrão e como o pode aplicar numa aplicação que utiliza ExpressJS. (1.5 val)

O Padrão **MVC (Model-View-Controller)** separa a aplicação em 3 áreas lógicas: - **Model**: Gere a estrutura dos dados e interage com a Base de Dados. No ExpressJS, aplica-se criando ficheiros dedicados a esquemas do **Mongoose**. - **View**: A parte visual que será enviada ao cliente. No ExpressJS, usam-se

Template Engines como o **EJS** (guardados na pasta *views*) para gerar HTML dinâmico com os dados. - **Controller**: O cérebro que liga as Views aos Models. No ExpressJS, aplica-se através de ficheiros na pasta *controllers* que processam os pedidos (req, res) definidos nas *Routes*, acedem ao Model e injetam o resultado no `res.render(view)`.

5. Considere a segurança de aplicações web. Indique de forma detalhada como podemos adicionar autenticação e autorização a uma aplicação web que use a framework para o frontend e backend. (3 val)

- 1. Autenticação (“Quem és?”):**
 - Backend recebe (via POST) as credenciais. Se válidas, assina um token **JWT** (JSON Web Token) e envia ao cliente.
 - O Angular guarda este token localmente (ex: `localStorage`).
- 2. Manter estado e proteger rotas (Frontend):**
 - No Angular, usa-se um **HTTP Interceptor** para apanhar cada novo pedido HTTP e injetar o token no header (`Authorization: Bearer <token>`).
 - Usa-se um **Guard** (`CanActivate`) no Router do Angular para impedir que browsers não autenticados abram páginas restritas.
- 3. Autorização (“O que podes fazer?”) (Backend):**
 - O ExpressJS utiliza um **middleware** próprio antes de rotas protegidas que faz o `jwt.verify()` ao token enviado pelo Angular, garantindo que não só o utilizador está autenticado, como tem as roles (permissões) necessárias na API.

6. Observe a figura 4 que representa uma aplicação web. Escreva o conteúdo do ficheiro `demo.html` de forma a o exemplo ficar funcional. (1.5 val)

(O código backend na imagem processa um `app.post('/result')` à espera de campos `username` e `password` para mostrar ‘Sucess’ ou ‘Failure’)

Para funcionar, precisamos de um Formulário HTML com o método e acção corretos:

```
<!DOCTYPE html>
<html>
<head><title>Página de Login</title></head>
<body>
<h2>Login</h2>
<!-- A ação tem de apontar para a rota do backend POST /result -->
<form action="/result" method="POST">
  <!-- Os atributos 'name' devem corresponder aos nomes procurados em req.body -->
  <label>Username:</label>
  <input type="text" name="username" required><br>

  <label>Password:</label>
  <input type="password" name="password" required><br>
```

```
<button type="submit">Entrar</button>
</form>
</body>
</html>
```

7. Observe o excerto de código de uma aplicação Angular presente na figura 5. Indique qual a sua utilidade e funcionalidade, como pode ser utilizados por componentes em Angular e que padrões de software estão a ser utilizados para gerir os pedidos HTTP. (2 val)

- **Utilidade/Funcionalidade:** É um **Angular Service** (RestService) criado com o decorador @Injectable(). A sua função é encapsular a comunicação via rede com uma API REST do backend para gerir “Produtos” (GET por ID, POST novo produto, GET lista).
- **Como é utilizado por componentes:** Os componentes não instanciam esta classe diretamente. Recorrem à **Injeção de Dependências (Dependency Injection)** pedindo-a no seu construtor (constructor(private restService: RestService) {}). De seguida, chamam as funções e fazem subscribe() aos dados.
- **Padrões de Software em uso:**
 - **Singleton:** Definido por providedIn: 'root', existe apenas uma instância partilhada por toda a app.
 - **Observer / Reactive Programming:** A comunicação HTTP no Angular utiliza **Observables** (RxJS), permitindo código assíncrono avançado através do padrão Publish/Subscribe em resposta a streams de dados.